

Module 1 — Semiconductor & Digital Foundations

Complete Lesson: From Silicon Atoms to AI Chip Arithmetic

Module duration: 2 weeks **What you will understand by the end:** How a grain of sand becomes a transistor, how transistors become logic gates, how logic gates become arithmetic circuits, and how those circuits perform the multiply-accumulate operation that powers every AI model on the planet.

Before You Begin: The Big Picture

Every time you run a neural network — whether it's GPT-4 generating text, Stable Diffusion drawing an image, or a ResNet-50 classifying a photo — the computation ultimately reduces to one thing happening billions of times per second inside a chip:

```
result = result + (input × weight)
```

This is the Multiply-Accumulate (MAC) operation. Everything else in this course — quantization, inference engines, FPGA accelerators, agentic systems — is about doing this operation faster, cheaper, and more efficiently.

This module answers the question: **at the most fundamental physical level, how does a computer actually do this?**

The answer starts with a silicon atom.

Topic 1.1 — Semiconductor Basics

What Is a Semiconductor?

Materials in our universe fall into three broad categories based on how well they conduct electricity:

Conductors (copper, gold, aluminium): electrons flow freely. Resistance is low. Touch two ends to a battery and current flows.

Insulators (rubber, glass, pure silicon dioxide): electrons are tightly bound. Almost no current flows, regardless of voltage applied.

Semiconductors (silicon, germanium, gallium arsenide): their conductivity sits between conductors and insulators — and crucially, their conductivity can be *controlled* by adding tiny amounts of impurities or applying electric fields.

This controllability is everything. It is what makes transistors possible, and transistors are what make computers possible.

Why Silicon?

Silicon (Si) is element 14 in the periodic table. Its outer shell has exactly 4 electrons — it wants 4 more to be "happy" (to complete its outer shell with 8 electrons).

In a crystal of pure silicon, each silicon atom shares its 4 outer electrons with 4 neighbouring silicon atoms. This creates a rigid covalent bond structure called the **crystal lattice**. At absolute zero temperature (0 Kelvin), all electrons are locked in these bonds — pure silicon is a perfect insulator.

At room temperature, thermal energy occasionally frees an electron from a bond, leaving behind a "hole" — a vacancy with a net positive charge. This is the origin of conductivity in silicon, and it is very weak.

But when we intentionally add specific impurities — a process called **doping** — we can dramatically and precisely change this conductivity.

Doping: Engineering Conductivity

N-type Silicon (Negative charge carriers)

Add a tiny amount of **phosphorus** (element 15, 5 outer electrons) to the silicon lattice. Phosphorus slots into the crystal lattice in place of a silicon atom. Four of its electrons form bonds with the surrounding silicon atoms — but the fifth electron has no bond partner. It is loosely held and easily freed to move through the crystal as a **free electron**.

Result: the silicon now has extra free electrons available to carry current. These free electrons are the **majority charge carriers** in N-type silicon. The phosphorus atoms are called **donor atoms** because they donate electrons.

P-type Silicon (Positive charge carriers)

Add a tiny amount of **boron** (element 5, 3 outer electrons) to the silicon lattice. Boron has only 3 outer electrons — one fewer than it needs to fill all four bond positions with neighbouring silicon atoms. This missing electron creates a **hole**: a position in the lattice that "wants" an electron.

A nearby electron can hop into this hole, but that creates a new hole where the electron came from. Holes move through the crystal as if they were positive charge carriers drifting in the opposite direction of electron flow.

Result: the silicon has extra holes available to carry current. Holes are the **majority charge carriers** in P-type silicon. The boron atoms are called **acceptor atoms**.

Important: Doped silicon is still electrically neutral overall. Adding phosphorus does not make the material positively charged — the phosphorus atom has 15 protons in its nucleus, still balancing its electrons. We have simply created extra mobile charge carriers.

The PN Junction: Heart of Electronic Devices

When you place N-type and P-type silicon in contact, something interesting happens at the

interface — the **PN junction**.

What happens at the junction: The excess electrons in the N-type region diffuse across the junction into the P-type region (attracted by the holes). The holes from the P-type region diffuse into the N-type region. This diffusion creates a region near the junction that is depleted of free charge carriers — the **depletion region**.

As electrons leave the N-type side, they leave behind positively charged phosphorus atoms (donor atoms that have lost their extra electron). As holes leave the P-type side, they leave behind negatively charged boron atoms. This builds up an **electric field** pointing from the N-type side to the P-type side, which eventually stops further diffusion.

The result is a built-in voltage across the depletion region (about 0.6–0.7V for silicon).

Forward Bias

Connect the positive terminal of a battery to the P-type side and the negative terminal to the N-type side. The external voltage pushes electrons toward the junction from the N-side and holes toward the junction from the P-side. When the applied voltage exceeds the built-in voltage (~0.6V for silicon), the depletion region collapses and current flows freely.

Forward biased PN junction = current flows = the diode conducts.

Reverse Bias

Flip the battery. Positive terminal to N-type, negative to P-type. The external voltage pulls electrons away from the junction on the N-side and pulls holes away on the P-side. The depletion region grows wider. Almost no current flows (just a tiny leakage current).

Reverse biased PN junction = almost no current = the diode blocks.

A PN junction is a **diode** — it conducts in one direction and blocks in the other.

MOSFET: The Transistor That Runs the World

Modern digital chips use MOSFETs: **Metal-Oxide-Semiconductor Field-Effect Transistors**. Every processor, GPU, and AI chip is built from billions of MOSFETs.

The MOSFET has three terminals:

- **Gate (G):** The control terminal. A voltage applied here controls whether current flows.
- **Source (S):** Where current enters (or where carriers originate).
- **Drain (D):** Where current exits (or where carriers go).

Between the gate metal and the semiconductor channel is a thin layer of silicon dioxide (SiO_2) — the oxide insulator. This means the gate is **electrically isolated** from the channel — virtually no DC current flows into the gate. The gate controls the channel through an electric field (hence "field-effect").

N-channel MOSFET (NMOS)

Structure: P-type silicon body with two N-type regions (source and drain) embedded in it. The gate sits above the channel region (the P-type silicon between source and drain), separated by the oxide.

With gate voltage = 0 (or below threshold): The P-type channel between the two N-type regions forms two back-to-back PN junctions. No current flows from drain to source. The MOSFET is **OFF** (open switch).

With gate voltage $\geq$ threshold voltage (V_{th} , typically 0.3–0.7V): The positive gate voltage attracts electrons to the surface of the P-type channel, forming a thin layer of N-type material (an **inversion layer** or **channel**). Now there is a continuous N-type path from source to drain. Current flows. The MOSFET is **ON** (closed switch).

In digital circuits: NMOS is ON (connects drain to source) when gate voltage is HIGH (logic 1). It is OFF when gate voltage is LOW (logic 0).

P-channel MOSFET (PMOS)

Structure: N-type silicon body with two P-type regions (source and drain). Complementary to NMOS.

With gate voltage = 0 (or above threshold): Channel forms, current flows. PMOS is **ON** when gate voltage is LOW (logic 0). It is **OFF** when gate voltage is HIGH (logic 1).

This is the opposite behaviour of NMOS — and this complementary behaviour is the foundation of CMOS.

CMOS: Why Modern Chips Are Efficient

CMOS (Complementary MOS) pairs an NMOS transistor with a PMOS transistor to implement any logic function. This pairing has one extraordinarily valuable property:

In steady state (inputs not switching), one of the two transistors is always OFF. There is no direct path from power supply to ground. Therefore, CMOS draws almost no static power.

The only time significant power is consumed is during switching — when the output changes state and the capacitance of the output wire charges or discharges. This is called **dynamic power** and we will analyse it carefully in Topic 1.5.

This is why CMOS dominated digital design from the 1980s onward and why essentially every chip you will ever work with — from a microcontroller to an A100 GPU — uses CMOS technology.

The CMOS NAND Gate: Your First Real Circuit

Let us build a NAND gate from transistors — this is the most fundamental circuit in digital design.

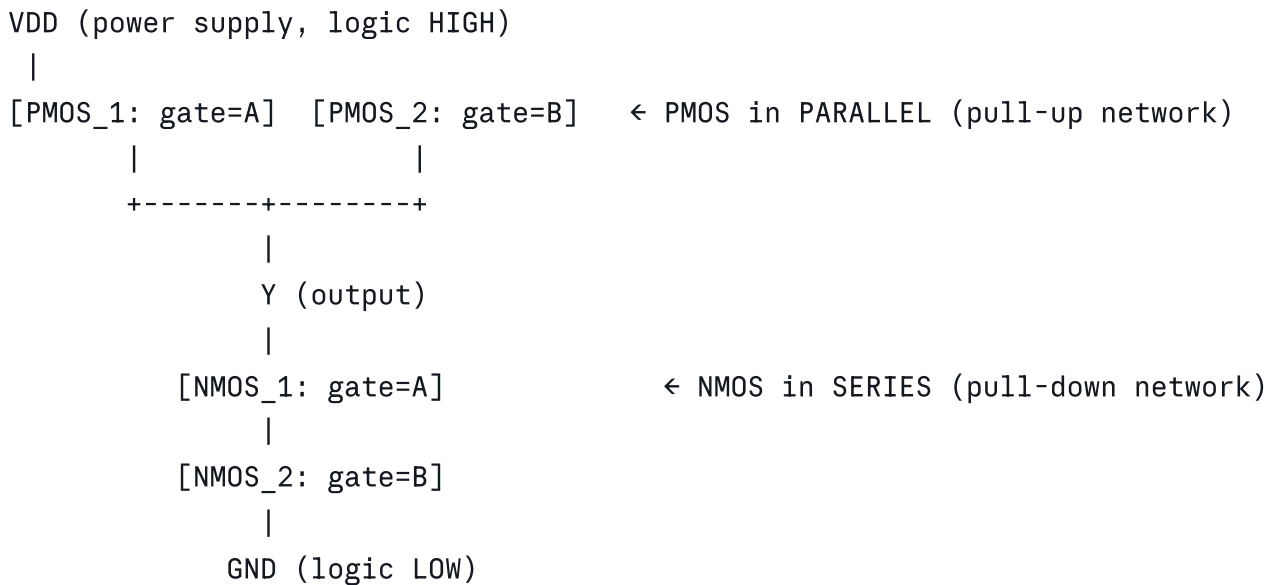
A NAND gate has two inputs (A and B) and one output (Y). The logic function is:

$$Y = \text{NOT}(A \text{ AND } B)$$

Truth table:

A	B	Y
0	0	1
0	1	1
1	0	1
1	1	0

CMOS NAND gate structure (4 transistors):



Analysis for each input combination:

A	B	PMOS_1	PMOS_2	NMOS_1	NMOS_2	Output Y
0	0	ON	ON	OFF	OFF	VDD (HIGH = 1)
0	1	ON	OFF	OFF	ON	VDD (HIGH = 1)
1	0	OFF	ON	ON	OFF	VDD (HIGH = 1)
1	1	OFF	OFF	ON	ON	GND (LOW = 0)

When A=1 and B=1: both PMOS are OFF (no path from VDD to output), both NMOS are ON (path from output to GND). Output is pulled LOW → logic 0. This is the only case where output is 0 — exactly what NAND requires.

For every other input combination: at least one PMOS is ON (pulling output HIGH) and the NMOS series path is broken (at least one NMOS is OFF). Output is HIGH → logic 1.

This is the most important circuit in this module. Understand it completely before moving on.

Key takeaway: we used 2 PMOS (in parallel for pull-up) and 2 NMOS (in series for pull-down).
For an N-input CMOS NAND gate: N PMOS in parallel + N NMOS in series = 2N transistors total.

Moore's Law and Its Limits

In 1965, Gordon Moore observed that the number of transistors on an integrated circuit had been doubling approximately every two years. This empirical observation — Moore's Law — held remarkably well for over 50 years.

The progression:

- Intel 4004 (1971): 2,300 transistors, 10,000 nm process
- Pentium (1993): 3.1 million transistors, 800 nm process
- Core i7 (2008): 731 million transistors, 45 nm process
- Apple M1 (2020): 16 billion transistors, 5 nm process
- Apple M4 (2024): 28 billion transistors, 3 nm process

Why are we running out of room?

As transistors shrink toward the atomic scale (silicon atoms are ~0.2 nm apart; current "2 nm" processes have gate lengths of ~8–12 nm physically), several physical limits emerge:

- **Quantum tunnelling:** Electrons tunnel through the gate oxide even when the transistor is supposed to be OFF, causing leakage current
- **Short-channel effects:** The gate loses control of the channel when it becomes too short
- **Heat density:** Packing more transistors per mm² generates more heat per unit area than can be removed
- **Manufacturing variability:** Small random variations in atomic positions cause transistors to behave differently

The industry response: instead of purely shrinking transistors, we now build **3D** — stacking chip layers (TSMC SoIC, Intel Foveros), packaging multiple chips together (chiplets), and designing more efficient architectures.

This is directly relevant to AI: the AI chip revolution is largely a response to the end of easy transistor scaling. We can no longer make CPUs twice as fast every two years. Instead, we build specialised accelerators (GPUs, TPUs, FPGAs) that use the available transistors more efficiently for the specific computations AI requires.

Topic 1.2 — Logic Gates & Boolean Algebra

The Seven Fundamental Gates

Every digital circuit — from a simple timer to a 100-billion-parameter LLM inference chip — is built from combinations of these seven fundamental logic gates.

NOT Gate (Inverter)

Input A $\rightarrow$ [NOT] $\rightarrow$ Output Y = $\bar{A}$ (A-bar, "NOT A")

Truth table:

A		Y
0		1
1		0

The simplest gate. Inverts its input. Built from: 1 PMOS + 1 NMOS (the fundamental CMOS inverter).

AND Gate

A $\neg$
[AND] $\rightarrow$ Y = A $\cdot$ B (A AND B)
B $\neg$

Truth table:

A		B		Y
0		0		0
0		1		0
1		0		0
1		1		1

Output is HIGH only when **both** inputs are HIGH. In CMOS: AND = NAND + NOT (4+2 = 6 transistors). This is why NAND, not AND, is the true primitive — it uses fewer transistors.

OR Gate

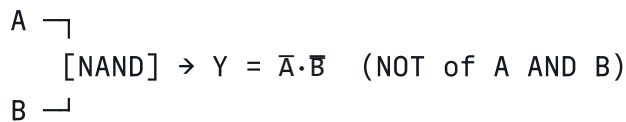
A $\neg$
[OR] $\rightarrow$ Y = A + B (A OR B)
B $\neg$

Truth table:

A	B	Y
0	0	0
0	1	1
1	0	1
1	1	1

Output is HIGH when **at least one** input is HIGH. In CMOS: OR = NOR + NOT (4+2 = 6 transistors).

NAND Gate (Not-AND)

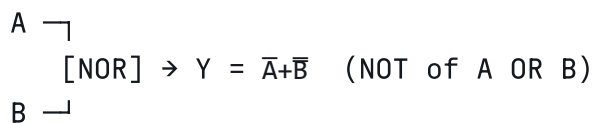


Truth table:

A	B	Y
0	0	1
0	1	1
1	0	1
1	1	0

Output is LOW **only** when both inputs are HIGH. As we built in Topic 1.1: 4 transistors. **This is the most important gate for CMOS implementation.**

NOR Gate (Not-OR)

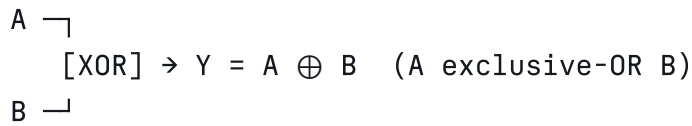


Truth table:

A	B	Y
0	0	1
0	1	0
1	0	0
1	1	0

Output is HIGH **only** when both inputs are LOW. Also 4 transistors (2 PMOS in series for pull-up, 2 NMOS in parallel for pull-down — opposite topology from NAND).

XOR Gate (Exclusive-OR)

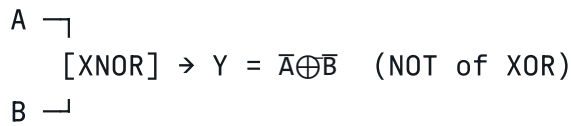


Truth table:

A	B	Y
0	0	0
0	1	1
1	0	1
1	1	0

Output is HIGH when inputs are **different**. This gate is the heart of binary addition: in a half adder, the sum bit is A XOR B. Requires 8-12 transistors in CMOS.

XNOR Gate (Exclusive-NOR)



Truth table:

A	B	Y
0	0	1
0	1	0
1	0	0
1	1	1

Output is HIGH when inputs are **equal**. Used in comparators: if A = B, then A XNOR B = 1 for every bit.

Boolean Algebra: The Mathematics of Logic

Boolean algebra gives us a mathematical framework to manipulate and simplify logic expressions. Every rule here has a direct consequence in circuit design — simpler expression = fewer gates = smaller chip = less power.

Fundamental Laws

Law	AND form	OR form
Identity	$A \cdot 1 = A$	$A + 0 = A$

Law	AND form	OR form
Null	$A \cdot 0 = 0$	$A + 1 = 1$
Idempotent	$A \cdot A = A$	$A + A = A$
Complement	$A \cdot \bar{A} = 0$	$A + \bar{A} = 1$
Double negation	$\bar{\bar{A}} = A$	—
Commutative	$A \cdot B = B \cdot A$	$A + B = B + A$
Associative	$(A \cdot B) \cdot C = A \cdot (B \cdot C)$	$(A + B) + C = A + (B + C)$
Distributive	$A \cdot (B + C) = A \cdot B + A \cdot C$	$A + (B \cdot C) = (A + B) \cdot (A + C)$
Absorption	$A \cdot (A + B) = A$	$A + A \cdot B = A$

De Morgan's Theorem: The Most Important Rule

Theorem 1: $\text{NOT}(A \text{ AND } B) = (\text{NOT } A) \text{ OR } (\text{NOT } B)$

In notation: $\boxed{(A \cdot B)^- = \bar{A} + \bar{B}}$

Theorem 2: $\text{NOT}(A \text{ OR } B) = (\text{NOT } A) \text{ AND } (\text{NOT } B)$

In notation: $\boxed{(A + B)^- = \bar{A} \cdot \bar{B}}$

Why this matters so much:

De Morgan's theorem tells you how to push a NOT through a gate — AND becomes OR (and vice versa), and all the inputs get negated.

In CMOS circuit design: NAND and NOR gates are cheaper to build than AND and OR gates. De Morgan's theorem lets you transform any expression into an equivalent form that uses only NAND or only NOR gates.

Example: Suppose you want to implement $Y = A \cdot B + C \cdot D$ (AND two pairs, then OR the results).

The straightforward implementation requires: two AND gates + one OR gate. In CMOS: $6 + 6 + 6 = 18$ transistors (because $\text{AND} = \text{NAND} + \text{NOT}$, $\text{OR} = \text{NOR} + \text{NOT}$).

Using NAND-only implementation with De Morgan's:

$$\begin{aligned}
 Y &= A \cdot B + C \cdot D \\
 &= ((A \cdot B + C \cdot D))^- && \text{[double negation]} \\
 &= ((A \cdot B)^- \cdot (C \cdot D)^-)^- && \text{[De Morgan's on the outer NOT]}
 \end{aligned}$$

So $Y = \text{NAND}(\text{NAND}(A,B), \text{NAND}(C,D))$ — three NAND gates, each 4 transistors = 12 transistors. Same result, 33% fewer transistors.

NAND and NOR: Universal Gates

NAND is a universal gate: any Boolean function can be implemented using only NAND gates.

Proof by construction (showing how to make all other gates from NAND):

- $\text{NOT } A = \text{NAND}(A, A) \rightarrow$ connect both inputs of a NAND to A
- $A \text{ AND } B = \text{NAND}(\text{NAND}(A, B), \text{NAND}(A, B)) \rightarrow$ NAND the NAND output with itself
- $A \text{ OR } B = \text{NAND}(\text{NAND}(A, A), \text{NAND}(B, B)) \rightarrow$ NOT each input, then NAND them (De Morgan's)

Since NOT, AND, OR are functionally complete (can express any Boolean function), and we can build NOT, AND, OR from NAND, NAND alone is sufficient.

Why this matters for hardware: A chip foundry only needs to characterise, optimise, and manufacture one type of gate — the NAND gate — to build any digital circuit. This simplifies chip design enormously.

Karnaugh Maps: Minimising Logic

A Karnaugh map (K-map) is a visual tool for minimising Boolean expressions. Given a truth table, it finds the simplest SOP (Sum of Products) expression with the minimum number of terms.

K-map rules:

- Arrange inputs so adjacent cells differ by exactly one variable (Gray code order)
- Group 1s in rectangles of size 1, 2, 4, 8, 16... (powers of 2)
- Groups can wrap around edges of the map
- Larger groups $\rightarrow$ fewer terms in the expression $\rightarrow$ simpler circuit
- Aim for the minimum number of the largest possible groups

4-variable K-map example:

		CD			
AB		00	01	11	10
00		1	1	0	1
01		0	0	0	0
11		0	0	0	0
10		1	1	0	1

Groups:

- Top-left and bottom-left corners: $A=0, C=0, D=0$ and $A=1, C=0, D=0$ (A doesn't matter, $C=0, D=0$) $\rightarrow$ group of 4: CD'

- Column CD=01: A=0,B=0,C=0,D=1 and A=1,B=0,C=0,D=1 (A doesn't matter, B=0, D=1, C=0) → group of 2: BCD... (continue analysis)

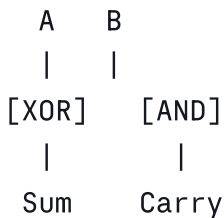
Don't care conditions: Some input combinations may never occur (e.g., BCD digit 1010-1111 are unused). Mark these as "X" — they can be treated as 0 or 1, whichever allows larger groupings.

In modern design, software tools (logic synthesizers like Yosys, Synopsys Design Compiler) perform this minimisation automatically on thousands of variables. Understanding K-maps teaches you what the tool is doing.

Topic 1.3 — Combinational Logic Circuits

Half Adder: Your First Useful Circuit

A half adder adds two 1-bit numbers (A and B) and produces a 2-bit result: a Sum bit and a Carry bit.



Boolean expressions:

- $\text{Sum} = A \oplus B$ (XOR)
- $\text{Carry} = A \cdot B$ (AND)

Truth table:

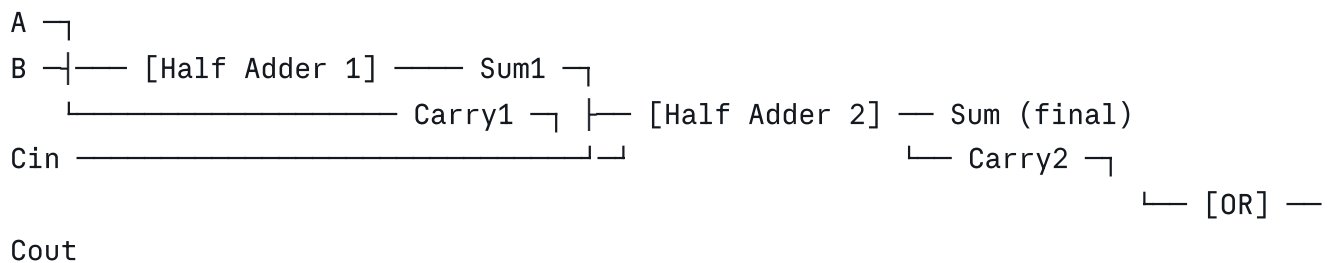
A	B	Sum	Carry
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

← 1+1=2 in decimal, which is "10" in binary: sum=0, carry=1

The half adder is called "half" because it cannot handle a carry input from a previous stage. For that, we need the full adder.

Full Adder: The Core of All Arithmetic

A full adder adds three 1-bit numbers: two data bits (A and B) plus a carry-in (Cin). It produces a Sum bit and a Carry-out (Cout).



Boolean expressions:

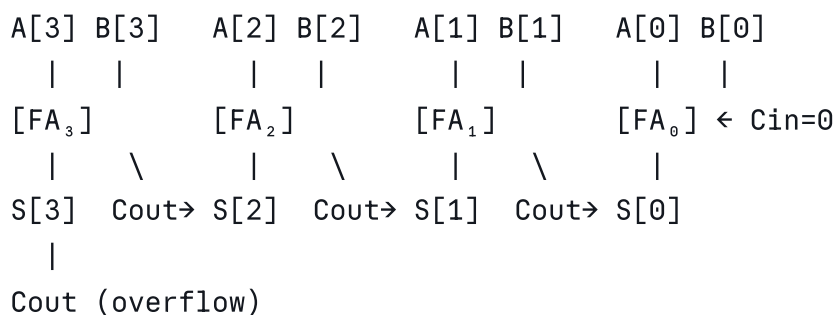
- $\text{Sum} = A \oplus B \oplus \text{Cin}$
- $\text{Cout} = A \cdot B + \text{Cin} \cdot (A \oplus B)$

Truth table:

A	B	Cin	Sum	Cout	
0	0	0	0	0	
0	0	1	1	0	
0	1	0	1	0	
0	1	1	0	1	← 0+1+1=2: sum=0, carry=1
1	0	0	1	0	
1	0	1	0	1	
1	1	0	0	1	← 1+1+0=2: sum=0, carry=1
1	1	1	1	1	← 1+1+1=3: sum=1, carry=1

Ripple Carry Adder: N-Bit Addition

To add two N-bit numbers, chain N full adders in sequence. The Carry-out of each stage becomes the Carry-in of the next stage.



Problem: propagation delay. Each full adder must wait for the carry from the previous stage before its output is stable. For a 64-bit adder, the critical path goes through 64 carry stages. The maximum clock frequency is limited by the time for a carry to ripple from bit 0 to bit 63.

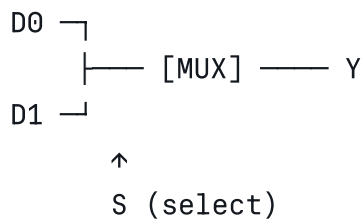
For a modern CPU running at 3-5 GHz, a 64-stage ripple carry adder would be far too slow. Real CPUs use **carry lookahead adders** that compute all carries in parallel, reducing the critical path to $O(\log N)$ levels.

Why does this matter for AI? The adder is inside every MAC unit in every AI chip. Intel, NVIDIA, and Google spend enormous effort designing the fastest possible adder circuits that fit within their power budget. The specific adder topology chosen — ripple carry for slow but small designs, Kogge-Stone for maximum speed, Brent-Kung for balanced area/speed — directly impacts the chip's TFLOPS rating.

Multiplexer (MUX): The Selector

A multiplexer selects one of N input signals and routes it to the output, based on a binary select signal.

2-to-1 MUX:



- If $S=0$: $Y = D0$
- If $S=1$: $Y = D1$

Boolean expression: $Y = (D0 \cdot \bar{S}) + (D1 \cdot S)$

Why the MUX is fundamental to FPGAs:

An FPGA's basic building block is a Look-Up Table (LUT). An N-input LUT is essentially a 2^n -to-1 MUX where:

- The 2^n data inputs are loaded with the desired truth table output values
- The N select lines are the N logic inputs
- The output is the function value for the current input combination

A 6-input LUT (standard in modern FPGAs) can implement any Boolean function of up to 6 variables. The LUT is just a 64-entry memory (64-to-1 MUX) with 64 programmable bits. This is why FPGAs can implement any digital circuit — they have thousands of these programmable truth tables.

Decoder: Address to Selection

A decoder takes an N-bit binary input and activates exactly one of 2^n output lines corresponding to that binary value.

2-to-4 decoder:

Input	A1	A0	Output	Y3	Y2	Y1	Y0	
	0	0		0	0	0	1	(Y0 active)
	0	1		0	0	1	0	(Y1 active)
	1	0		0	1	0	0	(Y2 active)
	1	1		1	0	0	0	(Y3 active)

Decoders are used in memory addressing: a 10-bit address decoder selects one of 1024 memory rows. Every time you access RAM, a decoder is activating the correct row of memory cells.

Propagation Delay and Critical Path

Every logic gate has a **propagation delay** — the time from when the inputs change to when the output settles to its correct value. Typical gate delays in modern CMOS:

- Inverter: ~10–50 picoseconds (ps)
- NAND, NOR: ~15–60 ps
- XOR: ~30–100 ps

In a combinational circuit, the **critical path** is the longest sequence of gate delays from any input to any output. This determines the minimum possible clock period (and therefore maximum clock frequency) of the circuit.

Example: A ripple carry adder's critical path passes through:

- One XOR gate (for the sum of the final stage)
- N chains of: [carry generate AND] + [carry propagate OR]

For a 16-bit adder with 50ps gate delay per stage: critical path $\approx 16 \times 100\text{ps} = 1.6 \text{ ns} \rightarrow$ maximum clock $\approx 625 \text{ MHz}$.

A carry lookahead adder reduces this to ~4 stages regardless of N $\rightarrow$ critical path $\approx 400 \text{ ps} \rightarrow 2.5 \text{ GHz}$.

Topic 1.4 — Sequential Logic & State Machines

Why We Need Memory: From Combinational to Sequential

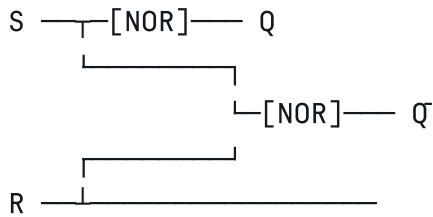
A combinational circuit's output depends only on its current inputs. Every time inputs change, the output immediately (after propagation delay) reflects the new input.

A **sequential circuit's** output depends on both its current inputs **and its history** — the sequence of past inputs. To store history, we need circuits that can remember their previous state. These are called **memory elements** or **storage elements**.

The most fundamental memory element in digital design is the **flip-flop**.

SR Latch: The First Memory Element

The SR (Set-Reset) latch is the simplest memory element, built from two cross-coupled NOR gates.



Behaviour:

- $S=1, R=0$: Q is SET to 1 ($Q=1, \bar{Q}=0$)
- $S=0, R=1$: Q is RESET to 0 ($Q=0, \bar{Q}=1$)
- $S=0, R=0$: Q **holds its previous value** — this is how memory works
- $S=1, R=1$: FORBIDDEN — both outputs forced to same value, contradicts $Q=\bar{Q}$

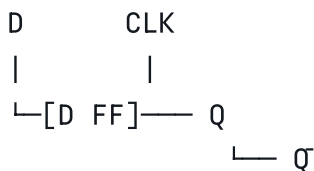
The SR latch demonstrates the key principle: feedback in a circuit creates memory. The cross-coupled NOR gates create a stable state that persists even after the inputs return to 0.

Problem with SR latch: It is *level-sensitive* — when S or R are active (logic 1), the output changes immediately. In a synchronous system where we want changes to happen only at specific times (on a clock edge), this is problematic.

D Flip-Flop: The Foundation of Synchronous Design

The **D flip-flop** (Data or Delay flip-flop) solves the SR forbidden state and adds clock control. It is the most important sequential element in digital design.

Behaviour: On the **rising edge** of the clock signal (when clock transitions from 0 to 1), the flip-flop captures the value present at its D input and holds it at the Q output until the next rising clock edge. Between clock edges, changes at D are ignored.



Truth table (edge-triggered):

D	CLK	Q (next state)
0	↑ (edge)	0
1	↑ (edge)	1
X	no edge	Q (unchanged)

Why this is powerful: In a synchronous digital system, every flip-flop updates simultaneously on the same clock edge. The combinational logic between flip-flops computes new values during one clock period, and then all flip-flops capture those values at once on the next clock edge. This creates a perfectly predictable, controllable computation pipeline.

Every register in a CPU, GPU, and FPGA is made from D flip-flops.

- A 64-bit register = 64 D flip-flops with shared clock and enable signals
- A register file (32 registers $\times$ 64 bits in a RISC-V CPU) = 2048 D flip-flops
- The entire state of a processor (all registers, PC, flags) is stored in flip-flops
- A typical GPU has millions of flip-flops just for its register file

Registers: Storing Multiple Bits

A **register** is simply N D flip-flops sharing a common clock and (optionally) a common enable signal.

```
D[7] D[6] D[5] D[4] D[3] D[2] D[1] D[0]
|   |   |   |   |   |   |   |
[FF] [FF] [FF] [FF] [FF] [FF] [FF] [FF]  ← 8 D flip-flops, shared clock
|   |   |   |   |   |   |   |
Q[7] Q[6] Q[5] Q[4] Q[3] Q[2] Q[1] Q[0]  ← 8-bit register output
```

An 8-bit register stores one byte. A 32-bit register stores one 32-bit integer or float. The accumulator register in a MAC unit is typically 32-48 bits wide (wider than the inputs, to accumulate many values without overflow).

Shift Registers: Moving Bits

A shift register chains flip-flops so that data shifts from one FF to the next on each clock edge.

```
DATA_IN → [FF0] → [FF1] → [FF2] → [FF3] → DATA_OUT (serial)
                ↓           ↓           ↓
                Q[1]       Q[2]       Q[3]       ← parallel outputs
```

On each clock edge, DATA_IN enters FF₀, FF₀'s old value enters FF₁, FF₁'s old value enters FF₂, and so on. After 4 clock cycles, 4 bits of serial input data are stored in parallel.

Shift registers appear everywhere:

- Serial-to-parallel converters (SPI, UART interfaces)
- Pipeline delay buffers in DSP
- Pseudo-random number generators (LFSR: Linear Feedback Shift Register)
- Data serializers in high-speed interfaces (PCIe, DDR)

Finite State Machines (FSMs): Logic That Remembers

A **Finite State Machine** is a system with:

- A finite number of **states** (stored in flip-flops)
- A set of **inputs**
- A set of **outputs**
- **Transition rules:** what state to go to next, given current state and inputs
- **Output rules:** what outputs to produce, given current state (and possibly inputs)

FSMs are the foundation of control logic in every digital system: CPU control units, memory controllers, communication protocol handlers, and AI accelerator sequencers all use FSMs.

Two types:

- **Moore machine:** outputs depend only on current state
- **Mealy machine:** outputs depend on current state AND current inputs (reacts faster, fewer states needed)

FSM Design Methodology

Step 1 — State diagram: Draw circles for states, arrows for transitions. Label arrows with the input condition that causes the transition. Label states (or arrows for Mealy) with outputs.

Step 2 — State encoding: Assign binary codes to each state. For N states, need $\text{ceil}(\log_2(N))$ flip-flops.

Step 3 — State table: Enumerate all state-input combinations and their next states and outputs.

Step 4 — Logic equations: Derive Boolean expressions for: next-state logic (combinational circuit that determines next state from current state + inputs) and output logic.

Step 5 — Implementation: Build with flip-flops (for state registers) and logic gates (for next-state and output logic).

Example: Traffic Light Controller (Moore FSM)

States: RED, GREEN, YELLOW

Inputs: Timer (T) — goes HIGH after the time period for each phase expires

Outputs: Light colour

State diagram:

```
RED —(T=1)→ GREEN —(T=1)→ YELLOW —(T=1)→ RED (cycle)
RED —(T=0)→ RED    (wait)
GREEN—(T=0)→ GREEN (wait)
YELLOW—(T=0)→ YELLOW (wait)
```

State encoding:

- RED = 00
- GREEN = 01
- YELLOW = 10

This FSM concept directly applies in AI accelerators: the MAC unit controller is an FSM that sequences through: IDLE → LOAD_WEIGHTS → LOAD_INPUT → COMPUTE → OUTPUT → IDLE states.

Topic 1.5 — Clocking, Timing & Power

The Clock Signal

The clock is the heartbeat of every synchronous digital system. It is a periodic square wave that oscillates between logic 0 and logic 1 at a fixed frequency.

Key clock parameters:

- **Period (T):** time for one complete cycle (HIGH + LOW) in nanoseconds
- **Frequency ($f = 1/T$):** cycles per second in GHz (1 GHz = 10^9 Hz, $T = 1$ ns)
- **Duty cycle:** fraction of the period that the clock is HIGH (typically 50%)
- **Rise time / Fall time:** time to transition between logic levels (should be much less than T)

Examples:

- Intel Core i9-13900K: 5.8 GHz → $T = 172$ ps
- Apple M4: 4.4 GHz → $T = 227$ ps
- NVIDIA H100 GPU: 1,980 MHz → $T = 505$ ps
- Xilinx PYNQ-Z2 FPGA: up to ~220 MHz → $T = 4.5$ ns

Setup Time, Hold Time, and Timing Violations

For a D flip-flop to correctly capture data, the input D must be stable for a minimum time around the clock edge:

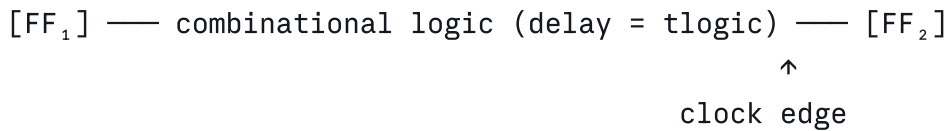
Setup time (t_{su}): D must be stable for at least t_{su} **before** the clock edge. This is the time the flip-flop's internal latch needs to sense the input.

Hold time (t_h): D must be stable for at least t_h **after** the clock edge. This prevents the input from changing before the flip-flop has fully latched the value.

Clock-to-Q delay (t_{cq}): Time from clock edge until Q output settles to new value.

The Critical Path Constraint

For a synchronous circuit with combinational logic between two flip-flops:



For correct operation: $T \geq t_{cq} + t_{logic} + t_{su}$

Rearranging: $f_{max} = 1 / (t_{cq} + t_{logic} + t_{su})$

The **critical path** is the combinational logic path with the maximum t_{logic} . It sets the maximum operating frequency.

Setup time violation: If the clock period is too short (frequency too high), the combinational logic hasn't settled before the setup time window closes. The flip-flop captures the wrong value (or enters metastability — an unstable state that can persist for an unpredictably long time).

Metastability is a real engineering risk: if a flip-flop enters a metastable state, its output slowly drifts toward a valid 0 or 1, but the time to resolve is statistically distributed. Worst case: the next logic stage sees an invalid intermediate voltage. Chip designers use careful timing analysis and synchronization circuits (double flip-flop synchronizers) to manage metastability.

Power Consumption in Digital Circuits

Understanding power is critical for AI hardware engineering. AI workloads are extraordinarily power-hungry: an NVIDIA H100 GPU has a TDP (Thermal Design Power) of 700W. A data center rack of 8 H100s dissipates 5,600W — hotter than a residential electric heater. Managing this power is a major engineering challenge.

Dynamic Power

Every time a logic gate switches (output changes state), it charges or discharges the capacitance connected to its output wire. This charge/discharge requires energy from the power supply.

Dynamic power formula:

$$P_{dynamic} = \alpha \times C \times V_{DD}^2 \times f$$

Where:

- **α (activity factor):** fraction of clock cycles in which the node switches ($0 \leq \alpha \leq 1$). A node that switches every cycle has $\alpha=1$; a node that rarely switches has $\alpha \approx 0$.
- **C (load capacitance):** capacitance of the output node, primarily from wire and input capacitance of connected gates. Measured in femtofarads (fF) to picofarads (pF) for individual nodes.
- **V_{DD} (supply voltage):** logic supply voltage. Currently 0.6–1.0V for leading-edge nodes (was 5V in the 1980s).

- **f (clock frequency):** transitions per second.

The V² relationship is the most important insight in this formula. Reducing VDD from 1.0V to 0.7V reduces dynamic power by $(0.7/1.0)^2 = 0.49$ — a 51% power reduction from a 30% voltage reduction. This is why voltage scaling is the most powerful power management technique.

Worked example — estimating GPU power:

An NVIDIA H100 has:

- ~80 billion transistors
- ~30 billion switching nodes per cycle (activity factor ~0.3 for typical workload)
- Average wire capacitance ~1 fF per node
- VDD ~0.85V
- f = 1.98 GHz

$$P \approx 0.3 \times (30 \times 10^9 \text{ nodes} \times 1 \times 10^{-15} \text{ F}) \times (0.85)^2 \times 1.98 \times 10^9 \text{ Hz}$$

$$P \approx 0.3 \times 30 \times 10^{-6} \text{ F} \times 0.72 \times 1.98 \times 10^9 \text{ Hz}$$

$$P \approx 0.3 \times 30 \times 10^{-6} \times 1.43 \times 10^9 \text{ W}$$

$$P \approx 13 \text{ W}$$

This is vastly less than the actual 700W. The difference is due to: wiring capacitance being much higher than 1 fF for global interconnects (can be 100× higher), I/O power, memory interface power, and the fact that switching activity is higher than estimated during compute workloads.

The estimate illustrates the formula — real GPU power analysis uses circuit simulators with accurate capacitance extraction from the physical layout.

Static Power (Leakage)

Even when no switching occurs, MOSFET transistors are not perfect switches. A small current leaks from drain to source even when the gate voltage is below the threshold voltage. This **subthreshold leakage current** is:

$$I_{\text{leak}} \propto \exp(-V_{\text{th}} / (n \times V_t))$$

Where V_{th} is the threshold voltage and $V_t = kT/q \approx 26 \text{ mV}$ at room temperature.

As transistors shrink, the gate oxide becomes thinner and gate leakage also increases (quantum tunnelling through the oxide). At 5nm and below, static power can be comparable to dynamic power.

Static power: $P_{\text{static}} = V_{\text{DD}} \times I_{\text{leak}} \times (\text{number of transistors})$

Power Management Techniques

Clock gating: The simplest and most effective technique. When a circuit block is idle (not being used), disconnect its clock. No clock → no switching → no dynamic power ($\alpha=0$ for that block).

Implemented with clock-gating cells: an AND gate between the global clock and the local clock enables.

In a large chip, fine-grained clock gating can reduce dynamic power by 20–40%. The clock distribution network itself consumes 20–40% of chip power in unconstrained designs — clock gating directly addresses this.

Power gating: Completely cut power supply to an idle block using a header (PMOS switch in series with VDD) or footer (NMOS switch in series with GND). Eliminates both dynamic and static power for the gated block.

Downside: state is lost when power is removed. The block must be "woken up" and reinitialized before use. Power gating has a wake-up time cost (typically a few hundred nanoseconds) and requires retention registers (special flip-flops that maintain state at ultra-low power) if state must be preserved.

DVFS (Dynamic Voltage and Frequency Scaling): Adjust both voltage and frequency based on workload demand. Running at half frequency → half dynamic power. Running at half voltage and half frequency → $0.5^2 \times 0.5 = 0.125 = 87.5\%$ power reduction.

DVFS is used in every modern processor, GPU, and smartphone SoC. The operating system or a power management controller continuously monitors workload and adjusts VDD and f to deliver the required performance at minimum power.

Thermal Design Power (TDP): The maximum sustained power a chip is specified to dissipate. The cooling system (heatsink, fan, liquid cooling) must handle this continuously. For AI training chips:

- NVIDIA H100 SXM: 700W TDP
- AMD MI300X: 750W TDP
- Intel Gaudi 3: 900W TDP
- Google TPU v4: ~170W (custom ASIC, more efficient)

AI inference chips are typically lower power:

- NVIDIA L4 (inference): 72W TDP
- Qualcomm AI 100: 75W TDP
- Groq LPU: ~150W

Topic 1.6 — Number Systems & Data Representation

Binary: The Language of Digital Circuits

A digital circuit's wires carry one of two voltages: HIGH (logic 1) or LOW (logic 0). All information — integers, fractions, text, images, neural network weights — must ultimately be

represented as sequences of these binary digits (bits).

Place values in binary:

2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0
128	64	32	16	8	4	2	1

Binary to decimal: Multiply each bit by its place value and sum.

Example: 10110101_2

$$= 1 \times 128 + 0 \times 64 + 1 \times 32 + 1 \times 16 + 0 \times 8 + 1 \times 4 + 0 \times 2 + 1 \times 1$$

$$= 128 + 32 + 16 + 4 + 1$$

$$= 181_{10}$$

Decimal to binary: Repeatedly divide by 2, record remainders (bottom to top gives binary representation).

$$181 \div 2 = 90 \text{ remainder } 1 \text{ (LSB)}$$

$$90 \div 2 = 45 \text{ remainder } 0$$

$$45 \div 2 = 22 \text{ remainder } 1$$

$$22 \div 2 = 11 \text{ remainder } 0$$

$$11 \div 2 = 5 \text{ remainder } 1$$

$$5 \div 2 = 2 \text{ remainder } 1$$

$$2 \div 2 = 1 \text{ remainder } 0$$

$$1 \div 2 = 0 \text{ remainder } 1 \text{ (MSB)}$$

Reading bottom to top: 10110101_2 ✓

Hexadecimal (base 16): A compact representation of binary — each hex digit represents exactly 4 bits (a nibble).

Hex digit	Decimal	Binary
0	0	0000
1	1	0001
...	...	...
9	9	1001
A	10	1010
B	11	1011
C	12	1100
D	13	1101
E	14	1110
F	15	1111

Example: $0xB5 \text{ (hex)} = 1011 \ 0101_2 = 181_{10}$

GPU memory addresses, model weight file formats, and hardware registers are all typically displayed in hexadecimal.

Sign-magnitude: Use one bit (MSB) for sign (0=positive, 1=negative), remaining bits for magnitude.

Problem: Two representations of zero (+0 = 00000000, -0 = 10000000). Addition hardware must handle the sign separately. Almost never used in modern computers.

One's complement: Negate a number by inverting all bits.

+5 = 00000101

-5 = 11111010 (invert all bits)

Problem: Still two zeros. Arithmetic is awkward (end-around carry).

Two's complement: Negate a number by inverting all bits and adding 1.

+5 = 00000101

Invert: 11111010

Add 1: 11111011 → this is -5 in two's complement

Check: $5 + (-5) = 00000101 + 11111011 = 100000000$ (carry out, 8-bit result is 00000000 = 0 ✓)

Why two's complement dominates: The addition circuit is identical for signed and unsigned numbers. There is exactly one representation of zero. The range for N-bit two's complement is $[-2^{(N-1)}, +2^{(N-1)}-1]$. For 8 bits: -128 to +127.

Sign extension: To extend an N-bit two's complement number to $M > N$ bits, replicate the sign bit (MSB) into all new high-order positions.

-5 in 8-bit: 11111011

-5 in 16-bit: 1111111111111011 (sign bit 1 extended)

+5 in 8-bit: 00000101

+5 in 16-bit: 0000000000000101 (sign bit 0 extended)

Fixed-Point Numbers

A fixed-point number has a binary point at a fixed position between integer and fractional bits.

Q-format notation: Q(m).(n) means m integer bits and n fractional bits.

Example: Q8.8 (16 bits total)

Bit 15..8 = integer part (2^7 to 2^0)

Bit 7..0 = fractional part (2^{-1} to 2^{-8})

Value of $\text{bit}[k] = 2^{(k-8)}$ where k is counted from LSB.

Example Q8.8 value: 0000001000000000₂

= 2^0 in the fractional part? No — let's index properly:

Bits 15-8 (integer): 00000010 = 2

Bits 7-0 (fraction): 00000000 = 0

Value = 2.0

Example: 0000000110000000₂

Integer bits: 00000001 = 1

Fraction bits: 10000000 = $2^{-1} = 0.5$

Value = 1.5

Why fixed-point matters for AI hardware: Fixed-point arithmetic uses the same addition circuits as integer arithmetic. Integer multipliers are simpler and smaller than floating-point multipliers. For AI inference, if model weights can be represented in Q4.4 or Q2.6 format without unacceptable accuracy loss, the hardware becomes dramatically simpler and more power-efficient.

FPGAs and custom ASIC accelerators for AI frequently use fixed-point arithmetic rather than floating-point. FINN (the FPGA AI framework we use in Module 8) generates fixed-point inference circuits with programmable precision.

Floating-Point: IEEE 754 Standard

For AI training and high-precision inference, we use floating-point numbers — a hardware implementation of scientific notation in binary.

A floating-point number has three components:

- **Sign (S):** 1 bit — 0 for positive, 1 for negative
- **Exponent (E):** biased binary integer encoding the power of 2
- **Mantissa (M):** fractional bits encoding the significant digits

Value = $(-1)^S \times 2^{(E - \text{bias})} \times 1.M$

The "1." is implicit — floating-point assumes the leading bit is always 1 (normalised form), so it doesn't need to be stored. This gives one extra bit of precision for free.

FP32 — IEEE 754 Single Precision

Bit 31: Sign (1 bit)

Bits 30-23: Exponent (8 bits), bias = 127

Bits 22-0: Mantissa (23 bits)

- Exponent range: -126 to +127 (biased values 1-254; 0 and 255 are special)
- Magnitude range: $\sim 1.18 \times 10^{-38}$ to $\sim 3.4 \times 10^{38}$
- Decimal precision: ~ 7 significant digits

Special values:

- Exponent=0, Mantissa=0: $\pm$ Zero

- Exponent=255, Mantissa=0: $\pm\text{Infinity}$
- Exponent=255, Mantissa $\neq$ 0: NaN (Not a Number) — result of $0/0$, $\infty-\infty$, etc.
- Exponent=0, Mantissa $\neq$ 0: Denormal (subnormal) numbers — very small values, lose the implicit leading 1

Example: Represent -6.5 in FP32

Step 1: Convert to binary: $6.5_{10} = 110.1_2$

Step 2: Normalise: 1.101×2^2

Step 3: Sign=1 (negative)

Step 4: Exponent = $2 + 127$ (bias) = $129 = 10000001_2$

Step 5: Mantissa = 101 followed by 20 zeros = 101000000000000000000000

Result: 1 10000001 101000000000000000000000

In hex: 0xC0D00000

FP16 — Half Precision

Bit 15: Sign (1 bit)
 Bits 14-10: Exponent (5 bits), bias = 15
 Bits 9-0: Mantissa (10 bits)

- Exponent range: -14 to +15 (biased 1-30; 0 and 31 special)
- Magnitude range: $\sim 5.96 \times 10^{-8}$ to 65,504
- Decimal precision: ~ 3 significant digits

Limitations of FP16 for AI:

- Maximum value of 65,504 causes overflow during training (gradient explosions)
- Reduced mantissa precision (10 bits vs 23) causes gradient underflow
- Requires loss scaling tricks during training

FP16 is widely used for inference — the reduced precision is acceptable once the model is trained.

BF16 — Brain Float 16 (Google's Innovation)

Bit 15: Sign (1 bit)
 Bits 14-7: Exponent (8 bits), bias = 127 ← Same as FP32!
 Bits 6-0: Mantissa (7 bits)

BF16 truncates the FP32 mantissa from 23 bits to 7 bits but keeps the same exponent range.

Why this was revolutionary:

- FP32 → BF16 conversion is trivial: just drop the bottom 16 bits of the mantissa
- Same dynamic range as FP32 → no overflow during training, no loss scaling needed
- 2× smaller than FP32 → 2× more values fit in GPU memory
- Hardware is simpler than FP32

BF16 is now the standard for AI training. NVIDIA Ampere and Hopper GPUs, Google TPUs, and AMD Instinct all support BF16 natively. A100 GPU: 312 TFLOPS in BF16 vs 77.6 TFLOPS in FP32.

FP8 — The Frontier Format

Two variants, both with 1 sign bit, 8 bits total:

E4M3: 4 exponent bits, 3 mantissa bits. Better precision, smaller range. Used for weights.

E5M2: 5 exponent bits, 2 mantissa bits. Better range, less precision. Used for gradients.

FP8 was introduced for AI training in NVIDIA H100 (Hopper architecture, 2022). It provides 2× the throughput of BF16 with careful scaling techniques. Pushing AI further toward integer arithmetic.

Integer Formats for AI Inference

For inference (not training), we can go even further:

Format	Bits	Values	Model size (7B params)	Use case
FP32	32	$\pm 3.4 \times 10^{38}$	28 GB	Training reference
BF16	16	$\pm 3.4 \times 10^{38}$	14 GB	Training, high-quality inference
FP16	16	$\pm 65,504$	14 GB	Inference
INT8	8	-128 to 127	7 GB	Efficient inference
INT4	4	-8 to 7	3.5 GB	Edge inference
INT2	2	-2 to 1	1.75 GB	Experimental
1-bit	1	{-1, 1}	~1 GB	BitNet (Microsoft, 2023)

INT8 quantization (introduced properly in Module 6): Map FP32 values to the INT8 range [-128, 127] using:

$$\text{quantized} = \text{round}(\text{float_value} / \text{scale}) + \text{zero_point}$$
$$\text{float_value} \approx \text{scale} \times (\text{quantized} - \text{zero_point})$$

Where scale and zero_point are computed from the distribution of the float values (calibration).

This is why Module 1 matters for AI inference: the choice of number format directly determines:

- Model memory footprint
 - Hardware compute throughput (INT8 MACs are 4× smaller than FP32 MACs)
 - Inference latency and cost
 - Accuracy (more precision = better accuracy, up to a point)
-

Topic 1.7 — Multiply-Accumulate (MAC) Operations

The MAC: One Operation to Rule Them All

A Multiply-Accumulate operation computes:

$$\text{accumulator} = \text{accumulator} + (A \times B)$$

This single operation is the building block of virtually every computation in deep learning:

Dot product: The dot product of two N-dimensional vectors A and B:

$$\text{result} = \sum_i (A[i] \times B[i]) \quad \text{for } i = 0 \text{ to } N-1$$

This is exactly N MAC operations. The dot product is the fundamental operation of a neuron: it computes the weighted sum of inputs.

Fully-connected layer: For a layer with input dimension M and output dimension N, a single input vector produces N output neurons, each computed as a dot product with M inputs. Total: $M \times N$ MACs.

Convolution: A 2D convolution with filter size $K \times K$, C_{in} input channels, C_{out} output channels, and output feature map $H \times W$:

$$\text{Total MACs} = K \times K \times C_{\text{in}} \times C_{\text{out}} \times H \times W$$

For ResNet-50's first conv layer (7×7 kernel, 3 in channels, 64 out channels, 112×112 output):

$$\text{MACs} = 7 \times 7 \times 3 \times 64 \times 112 \times 112 = 1.18 \text{ billion MACs}$$

Transformer attention: For one attention head with sequence length N and dimension d_k :

$$\text{Attention MACs} = 2 \times N^2 \times d_k \quad (\text{for } QK^T \text{ and then } \times V)$$

For Llama-3-8B (seq=4096, d_model=4096, 32 heads, d_k=128):

Per-head attention MACs = $2 \times 4096^2 \times 128 = 4.3$ billion MACs
All heads per layer = $32 \times 4.3\text{B} = 137$ billion MACs
All 32 layers = 4.4 trillion MACs (just for attention)

This is why AI chips need to be optimised for MACs above everything else.

Hardware Implementation of a MAC Unit

The Integer MAC Unit

A hardware MAC unit consists of:

1. Multiplier circuit: Takes two N-bit inputs and produces a 2N-bit product. An 8×8 multiplier produces a 16-bit result.

The simplest multiplier is the **array multiplier**: generate N copies of the multiplicand (shifted left by 0, 1, 2, ..., N-1 positions based on the corresponding bit of the multiplier), then add all partial products using a tree of adders.

For an 8×8 multiplier:

- 8 partial products, each 8 bits wide, shifted by 0-7 positions
- Added with a Wallace tree (reduces N numbers to 2 using carry-save adders, then final carry-propagate adder)
- Delay: $O(\log N)$ levels in the Wallace tree

2. Adder (accumulator adder): Adds the product to the running accumulator value.

3. Accumulator register: A flip-flop register that stores the running sum. Must be wider than the product (typically product_width + guard_bits) to prevent overflow during accumulation. For 8×8 MACs: 16-bit products, 32-bit accumulator (supports $2^{16} = 65,536$ accumulations before overflow).

4. Control logic (FSM): Sequences the operations: load → multiply → add → store.

Clock cycle 1: Load A[0] and B[0] into input registers
Clock cycle 2: Multiply → partial product available
Clock cycle 3: Add to accumulator
Clock cycle 4: Load A[1] and B[1] ...
(for a non-pipelined MAC)

Pipelined MAC

In a pipelined MAC, the multiply and add operations overlap:

Cycle: 1 2 3 4 5 6 7

Load: A0B0 A1B1 A2B2 A3B3 ...

Mult: A0B0 A1B1 A2B2 A3B3

Add: A0 A1 A2 A3

After the initial pipeline fill (latency = pipeline depth), the MAC produces one result per cycle (throughput = 1 MAC/cycle), regardless of how many cycles each stage takes.

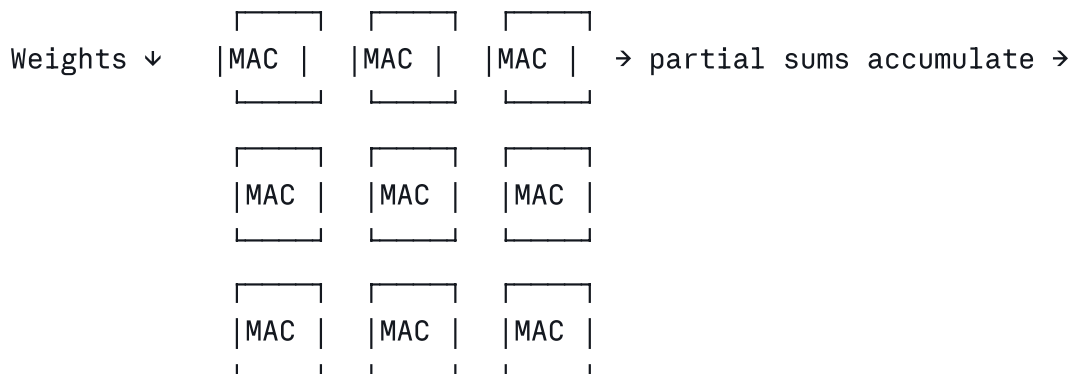
Latency vs throughput: Pipelining increases throughput (MACs per second) at the cost of latency (cycles to get the first result). For matrix multiply, where we need millions of MACs, throughput is what matters. The pipeline fills once and then runs at full speed.

Systolic Arrays: MACs That Talk to Each Other

In a naive matrix multiply implementation, weights are fetched from memory for each MAC operation. For large matrices, this creates a massive memory bandwidth bottleneck.

The systolic array solves this by arranging MAC units in a 2D grid and passing data between adjacent MACs directly — no memory access between MACs.

Data flows →



- Input activations flow LEFT to RIGHT, one step per cycle
- Weights flow TOP to BOTTOM, one step per cycle
- Each MAC multiplies its weight and input, adds to accumulating partial sum, passes both to neighbours

Google TPU v1 (2016): 256×256 systolic array = 65,536 MACs

- At 700 MHz: $65,536 \times 2 \times 700 \times 10^6 = 91.8$ TOPS (92 Tera Operations Per Second)
- Weights are loaded into the systolic array once and flow through — each weight is used 256 times
- This reuse eliminates the need to re-fetch weights from memory for every input

The systolic array is the hardware insight that enabled Google to train large language models efficiently in 2016 — before NVIDIA's Tensor Cores existed.

NVIDIA Tensor Cores

Introduced in Volta (2017), Tensor Cores perform a 4×4 matrix multiply-accumulate in a single "instruction":

$$D = A \times B + C \quad \text{where } A, B, C, D \text{ are } 4 \times 4 \text{ matrices}$$

This is 4×4×4 = 64 MACs per Tensor Core per clock cycle.

Tensor Core evolution:

GPU Architecture	Year	Precision	Throughput per SM
Volta (V100)	2017	FP16/FP32	8 TFLOPS per SM
Ampere (A100)	2020	BF16/TF32/INT8/FP8	77.6 TFLOPS/GPU
Hopper (H100)	2022	FP8/BF16/INT8	989 TFLOPS/GPU (sparse)
Blackwell (B200)	2024	FP4/FP8	~4.5 PFLOPS/GPU

The performance gap between CUDA cores and Tensor Cores is enormous — Tensor Cores are not just faster, they enable a fundamentally different computation model where large matrix operations are the primitive, not scalar multiply-add.

TOPS: Measuring AI Performance

TOPS = Tera Operations Per Second = 10¹² operations per second.

For a chip with M MAC units running at frequency f:

$$\text{TOPS} = M \times 2 \times f / 10^{12}$$

(Factor of 2 because each MAC performs 2 operations: one multiply and one add.)

Example: NVIDIA H100 SXM

- 4th-gen Tensor Cores: 528 TFLOPS in TF32 (sparse: 989 TFLOPS)
- But this is FLOPS, not integer TOPS
- In INT8: 3,958 TOPS (sparse)

The TOPS lie: Published TOPS ratings are under ideal conditions (maximum sparsity, no memory transfers, 100% utilisation). Real-world utilisation for LLM inference is often 30–60% of theoretical TOPS due to memory bandwidth bottlenecks.

This is the roofline model — which we will explore deeply in Module 2. A chip's effective performance for a given workload is limited by either:

- Peak TOPS (compute bound), or
- Memory bandwidth $\times$ arithmetic intensity (memory bound)

For LLM inference, especially for small batch sizes, the workload is almost always **memory-bandwidth bound** — the GPU's TOPS are sitting idle waiting for weights to arrive from VRAM.

FLOPS of major models:

Model	Parameters	Approx FLOPs per token (inference)
GPT-2 (1.5B)	1.5B	$\sim 3 \times 10^{12}$ FLOPS
Llama-3-8B	8B	$\sim 16 \times 10^{12}$ FLOPS
Llama-3-70B	70B	$\sim 140 \times 10^{12}$ FLOPS
GPT-4 (est.)	$\sim 1.8T$	$\sim 3,600 \times 10^{12}$ FLOPS

Rule of thumb: $\text{FLOPs} \approx 2 \times \text{parameters} \times \text{sequence length}$ (for inference, one forward pass).

For a 7B model on an H100 at 3,958 TOPS (INT8): theoretical time per token:

$$14 \times 10^{12} \text{ FLOPs} / (3.958 \times 10^{12} \text{ FLOPs/second}) \approx 3.5 \text{ ms/token}$$

At this rate: ~ 286 tokens/second — consistent with observed vLLM benchmarks (~ 200 – 400 tok/s depending on batch size, sequence length, and quantization level).

Module 1 Summary: What You Now Understand

Let us trace the complete path from atoms to AI computation:

Silicon atom $\rightarrow$ doped to create N-type and P-type regions $\rightarrow$ PN junction enables controlled current flow $\rightarrow$ **MOSFET transistor** as a voltage-controlled switch

Transistors combined in CMOS pairs $\rightarrow$ **Logic gates** (NAND, NOR, XOR...) that implement Boolean functions with near-zero static power

Logic gates combined $\rightarrow$ **Combinational circuits**: adders, multiplexers, decoders — circuits that compute

D flip-flops added $\rightarrow$ **Sequential circuits**: registers, counters, FSMs — circuits that remember and sequence

Binary representation → two's complement integers → IEEE 754 floating point → FP16/BF16 → INT8/INT4 for AI

Full adder → **N-bit adder** → **Multiplier** → **MAC unit** → **MAC array** → **Systolic array** → the core compute engine of every AI chip

TOPS rating → determined by $\text{MAC count} \times \text{frequency} \times \text{precision}$ → limited by power budget ($V^2 \times f \times C \times \alpha$) → why lower precision (INT8, INT4) enables more MACs per watt

This is the entire story of Topic 1. Every concept in the remaining nine modules builds on this foundation.

Key Formulas to Memorise

MOSFET:

NMOS ON when $V_{GS} \geq V_{th}$ (gate HIGH)

PMOS ON when $V_{GS} \leq -|V_{th}|$ (gate LOW)

CMOS NAND: 2 PMOS parallel + 2 NMOS series = 4 transistors

Boolean:

De Morgan: $(A \cdot B)^{\sim} = \bar{A} + \bar{B}$ $(A + B)^{\sim} = \bar{A} \cdot \bar{B}$

Full adder:

Sum = $A \oplus B \oplus C_{in}$

Cout = $AB + C_{in}(A \oplus B)$

Timing:

$f_{max} = 1 / (t_{cq} + t_{critical_path} + t_{su})$

Power:

$P_{dynamic} = \alpha \times C \times V^2 \times f$ [V^2 is the key insight]

Two's complement negation:

$-X = \sim X + 1$ (invert all bits, add 1)

IEEE 754 FP32:

Value = $(-1)^S \times 2^{(E-127)} \times 1.M$

MAC:

$acc = acc + (A \times B)$

$TOPS = MAC_count \times 2 \times frequency / 10^{12}$

Recommended Resources for Topic Depth

Semiconductors and CMOS:

- Sedra & Smith, *Microelectronic Circuits* — Chapters 1, 4, 5 (deep on MOSFETs)
- Neil Weste & David Harris, *CMOS VLSI Design* — Chapters 1-3 (free PDF available)

Digital Logic:

- David Harris & Sarah Harris, *Digital Design and Computer Architecture* — Chapters 1-4
- Neso Academy YouTube series: "Digital Electronics" — excellent short videos for each gate

Number Systems and IEEE 754:

- "What Every Computer Scientist Should Know About Floating-Point Arithmetic" — Goldberg (1991), ACM Computing Surveys — free online, canonical reference
- IEEE 754-2019 Standard (available from IEEE, also summarised in Wikipedia)

AI Hardware and MACs:

- Onur Mutlu, *Computer Architecture* lecture notes (ETH Zurich, free online)
- "In-Datacenter Performance Analysis of a Tensor Processing Unit" — Jouppi et al. (2017) — the Google TPU paper that introduced systolic arrays to AI

Simulation Tools:

- Logisim Evolution: <https://github.com/logisim-evolution/logisim-evolution> (free, runs on Java)
- Digital (hneemann): <https://github.com/hneemann/Digital> (alternative, very good)

Next: Module 1 Lab Notebooks — Project 1-A (Logic Gate Simulator), 1-B (Number Format Converter), 1-C (MAC Unit in Python + Logisim), 1-D (AI Chip FLOP Counter)

Module 2: Computer Architecture & Accelerators — how CPUs, GPUs, and FPGAs are built from these digital foundations, and how the roofline model predicts real AI performance.